

[Наименование образовательной организации]

[Факультет или институт]

[Кафедра]

ОТЧЁТ

по лабораторной работе № 1

Классификация

по дисциплине

[Наименование дисциплины]

Выполнил: [ФИО студента]

Группа: [Номер группы]

Проверил: [ФИО преподавателя]

Подпись: _____

Дата: _____

[Город] 2026

СОДЕРЖАНИЕ

1 Цель и задачи работы	3
2 Краткие теоретические сведения	4
3 Исходные данные и среда выполнения	5
4 Подробный процесс выполнения	6
5 Визуализация предварительной обработки	8
6 Данные и методика	10
7 Лучшие параметры	11
8 Оценка на тестовой выборке	12
9 Визуализация дерева	13
ЗАКЛЮЧЕНИЕ	15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	16
ПРИЛОЖЕНИЕ А	17
ПРИЛОЖЕНИЕ Б	20

1 Цель и задачи работы

Получить и закрепить навыки предварительной обработки данных и применения методов машинного обучения для решения задачи классификации дефолта по кредиту.

Для достижения цели поставлены следующие задачи:

- Исследовать распределения и связи признаков, удалить пропуски и дубликаты.
- Разделить данные на обучение и тест, закодировать категории и масштабировать числа.
- Обучить KNN, SVM и дерево решений, подобрать параметры по кросс-валидации.
- Вычислить precision, recall, F1 и ROC-AUC; построить матрицы ошибок, ROC-кривые и дерево.
- Сравнить модели и сформулировать вывод с учётом пропущенных дефолтов.

2 Краткие теоретические сведения

Классификация относится к обучению с учителем: по вектору признаков x модель определяет дискретную метку y . В данной работе положительный класс означает дефолт по кредиту. Обучение выполняется на размеченных наблюдениях, качество проверяется на данных, которые не использовались при выборе параметров.

KNN определяет класс по ближайшим объектам обучающей выборки. Число соседей k задаёт степень сглаживания границы; веса по расстоянию усиливают вклад близких объектов. SVM строит разделяющую поверхность с большим зазором между классами. Параметр C задаёт штраф за ошибки, а ядро RBF позволяет моделировать нелинейную границу. Дерево решений последовательно проверяет условия на признаки. Ограничение глубины и минимального размера листа уменьшает сложность модели [3].

Для матрицы ошибок TP - найденные дефолты, FP - ложные предсказания дефолта, FN - пропущенные дефолты, TN - верные ответы класса без дефолта. $\text{Precision} = TP / (TP + FP)$; $\text{recall} = TP / (TP + FN)$; $F1 = 2 \cdot \text{precision} \cdot \text{recall} / (\text{precision} + \text{recall})$. ROC-кривая показывает соотношение доли найденных положительных объектов и доли ложных срабатываний при изменении порога. ROC-AUC оценивает ранжирование объектов и не требует, чтобы оценка SVM была вероятностью.

Масштабы числовых признаков влияют на расстояния KNN и SVM. Стандартизация вычисляет $z = (x - \mu) / \sigma$. Параметры μ и σ определяются только по тренировочным объектам. OneHot-кодирование представляет категории индикаторными столбцами. Кросс-валидация многократно разделяет тренировочную выборку на обучающую и проверочную части; тест сохраняется для окончательной оценки.

3 Исходные данные и среда выполнения

Использован Credit Risk Dataset автора Lao Tse с Kaggle, лицензия CC0 [2]. Таблица содержит 32 581 запись и 12 столбцов. В `person_emp_length` отсутствуют 895 значений, в `loan_int_rate` - 3116. После удаления строк с пропусками и 137 дубликатов осталось 28 501 наблюдение. Число удалённых строк с пропусками равно 3943, поскольку пропуски в двух столбцах могут встречаться в одной строке.

Числовые столбцы описывают возраст, доход, стаж, сумму кредита, ставку, отношение кредита к доходу и длительность кредитной истории. Категории описывают жильё, цель кредита, категорию кредита и историю дефолтов. `loan_status` - бинарный исход кредита. Исходный файл находится в `data/raw/credit_risk_dataset.csv` и сохраняется без изменений.

Расчёты выполнены средствами Python 3.12.14 и `scikit-learn` 1.9.1. `pandas` используется для таблиц, `NumPy` - для массивов, `matplotlib` и `seaborn` для графиков. Точные версии зависимостей сохранены в `requirements-lock.txt` соответствующей лабораторной. Аномалии возраста и стажа не удалялись; они учитываются как ограничение интерпретации результатов.

4 Подробный процесс выполнения

Шаг 1. CSV прочитан с помощью `pandas.read_csv`. Функция `visualize` строит гистограммы числовых признаков, ящики с усами по классам дефолта и диаграммы рассеяния. Столбец `loan_status` отделён от 11 признаков. Загруженный файл хранится без изменений.

Шаг 2. `dropna` удаляет строки с хотя бы одним пропуском; `drop_duplicates` оставляет по одной копии одинаковых строк. Для исследовательского CSV выполнена min-max нормализация: $(x - \min) / (\max - \min)$. Модели этот CSV не используют: их `StandardScaler` обучается отдельно внутри `Pipeline`, чтобы не переносить информацию теста в обучение. Корреляция Пирсона рассчитана только для числовых столбцов.

Шаг 3. `train_test_split` выделяет 20% данных в тест с `stratify=loan_status` и `random_state=42`. В тренировочной выборке 17 850 объектов класса 0 и 4950 класса 1; в тесте 4463 и 1238 соответственно. Для каждой модели создан `Pipeline` из преобразований и классификатора.

Шаг 4. `GridSearchCV` проверяет 6 конфигураций KNN, 6 SVM и 16 деревьев на трёх стратифицированных фолдах. Выбор выполняется по F1 класса дефолта. Лучшая конфигурация каждого метода переобучается на всех 22 800 тренировочных строках.

Шаг 5. На 5701 тестовой строке рассчитаны метрики и индивидуальные предсказания. Для ROC-AUC KNN и дерево используют вероятность класса 1, SVM - `decision_function`. Сохранены результаты всех комбинаций, матрицы ошибок, ROC-кривые и первые четыре уровня дерева. Полная глубина выбранного дерева равна 8.

Шаг 6. Файл `view_results.py` собирает сохранённые метрики и изображения в локальную HTML-страницу. Ниже приведён снимок этой фактической страницы, подтверждающий наличие результатов. Для повторного обучения используется параметр `--rebuild`. Полный вычислительный код приведён в приложениях А и Б.

Команда просмотра из корня проекта: `.\venv\Scripts\python Лаба1/view_results.py`. Для повторного расчёта к команде добавляется `--rebuild`.

На рисунке 1 представлен результат этапа: снимок локальной страницы с результатами лабораторной.

Лабораторная №1. Классификация

Исходных строк: 32581; удалено с пропусками: 3943; дубликатов: 137; осталось: 28501.

Обучение: 22800 строк; тест: 5701. Подбор параметров: 3 фолда CV, критерий F1 класса «дефолт».

Метрики

Precision, recall и F1 относятся к классу 1 — дефолту.

model	cv_f1	precision	recall	f1	roc_auc
KNN	0.7099	0.8815	0.5889	0.7061	0.8771
SVM	0.7791	0.9120	0.6616	0.7669	0.9010
DecisionTree	0.8030	0.9759	0.6858	0.8055	0.9092

Выбрана по CV F1: DecisionTree.

Дерево нашло 849 из 1238 дефолтов и пропустило 389. Выбросы пока сохранены.

- ▶ KNN: параметры и метрики обоих классов
- ▶ SVM: параметры и метрики обоих классов
- ▶ DecisionTree: параметры и метрики обоих классов

Матрицы ошибок

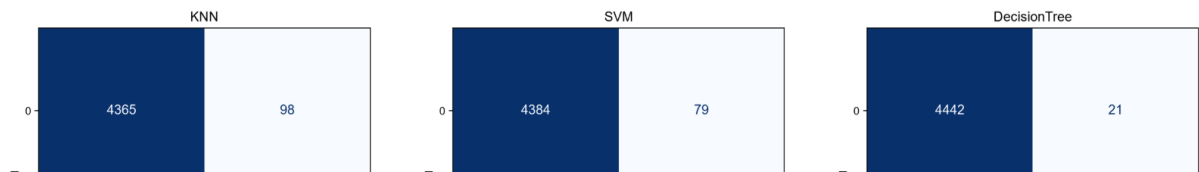


Рисунок 1 - Снимок локальной страницы с результатами лабораторной

5 Визуализация предварительной обработки

На рисунке 2 представлен результат этапа: гистограммы очищенных числовых признаков.

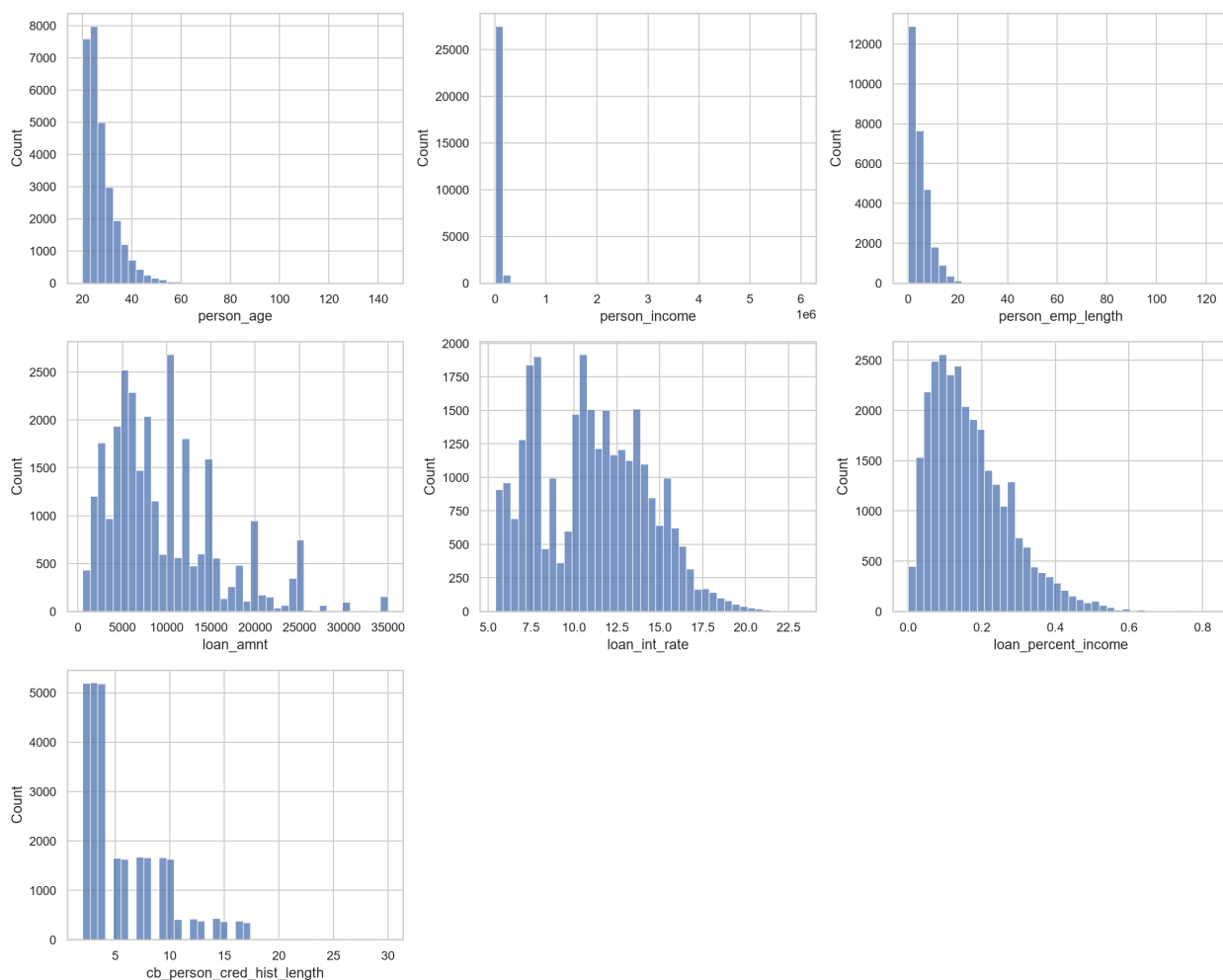


Рисунок 2 - Гистограммы очищенных числовых признаков

На рисунке 3 представлен результат этапа: ящики с усами признаков по классу дефолта.

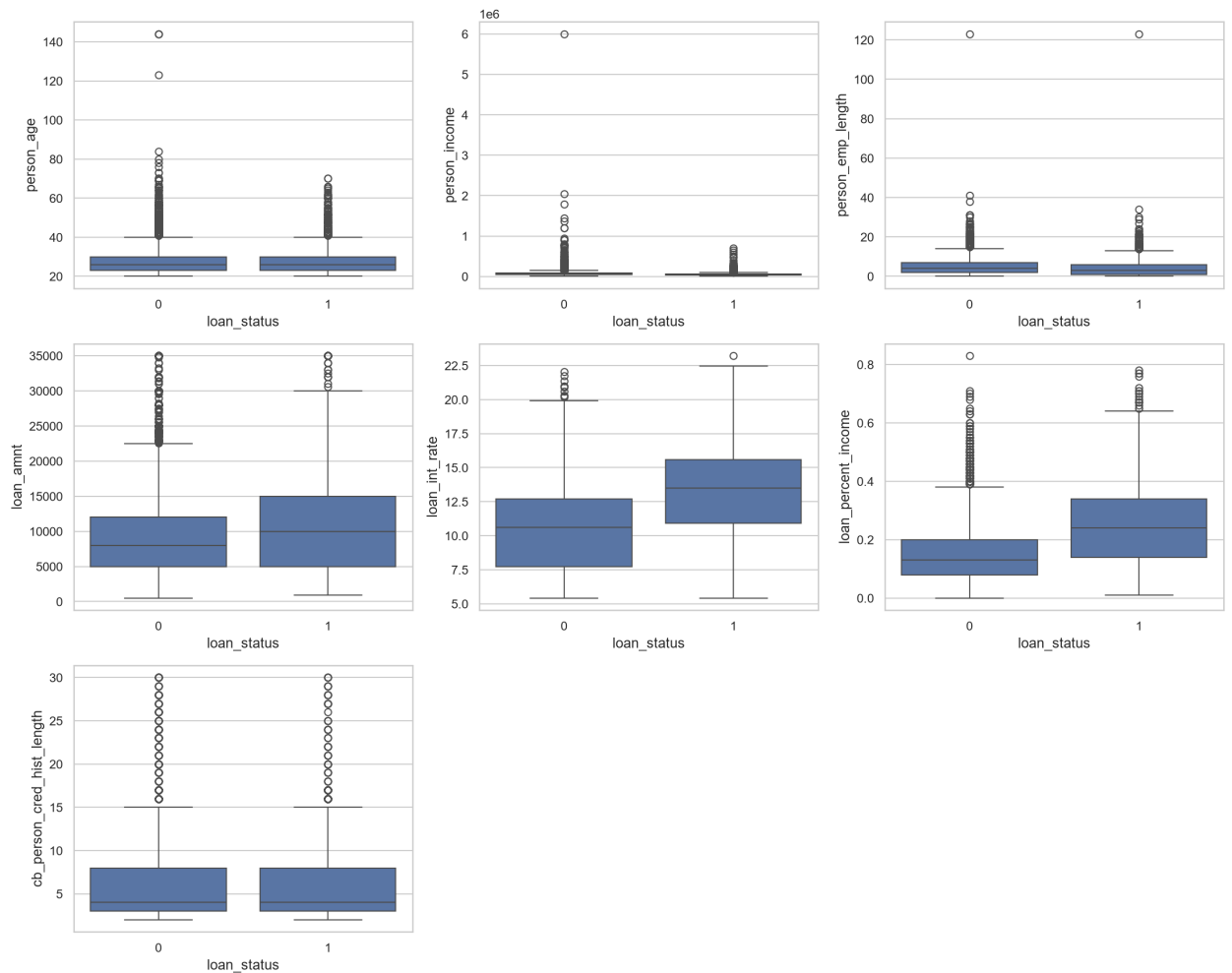


Рисунок 3 - Ящики с усами признаков по классу дефолта

На рисунке 4 представлен результат этапа: диаграммы рассеяния с классами дефолта.

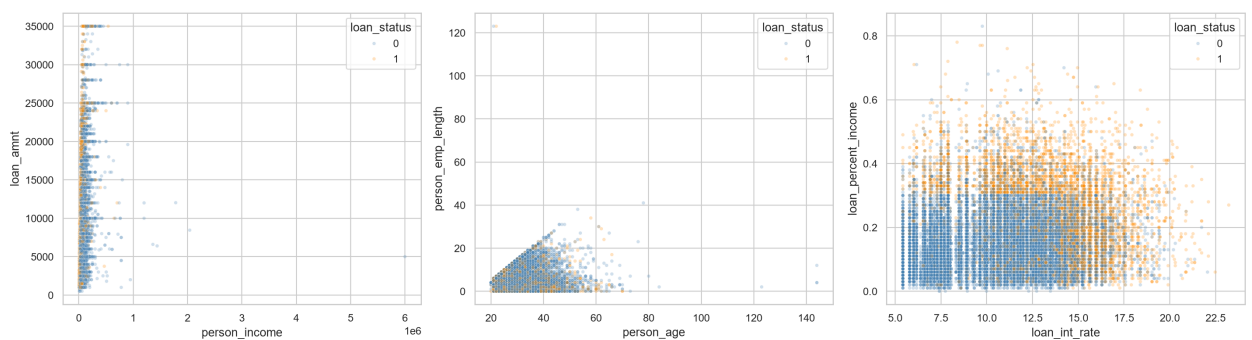


Рисунок 4 - Диаграммы рассеяния с классами дефолта

На рисунке 5 представлен результат этапа: матрица корреляций числовых столбцов.



Рисунок 5 - Матрица корреляций числовых столбцов

Корреляция возраста и кредитной истории около 0.86. Наибольшая положительная линейная связь с дефолтом среди этих числовых столбцов наблюдается у `loan_percent_income` (около 0.38) и `loan_int_rate` (около 0.34). Это описательные связи, а не доказательство причинности или формальной значимости.

6 Данные и методика

Credit Risk Dataset: 32 581 строка, 11 признаков и целевой столбец. Удалены 3 943 строки с пропусками и затем 137 дубликатов. Осталось 28 501 наблюдение. Исходные данные не изменены; аномальные значения возраста и стажа пока сохранены, поэтому выводы относятся к этой версии очистки.

Обучающая выборка: 22 800 строк (17 850 класса 0, 4 950 класса 1).
Тестовая выборка: 5 701 строка (4 463 класса 0, 1 238 класса 1). Разделение стратифицированное, доля теста 20%, random_state=42.

Числовые признаки стандартизированы, категории закодированы OneHotEncoder. Преобразования включены в Pipeline и обучаются внутри каждого тренировочного фолда; тестовая выборка не используется для выбора параметров. GridSearchCV использует 3 стратифицированных фолда и максимизирует F1 класса 1. Всего проверены 6 комбинаций KNN, 6 SVM и 16 дерева (84 обучения на фолдах, затем по одному обучению лучшей конфигурации на всей тренировочной выборке).

7 Лучшие параметры

В таблице 1 приведены результаты сравнения моделей.

Таблица 1 - Результаты сравнения моделей

Модель	Параметры	Средний F1 CV
KNN	n_neighbors=15, weights=distance	0.7099
SVM	kernel=rbf, C=10, gamma=scale	0.7791
Дерево решений	max_depth=8, min_samples_leaf=5, class_weight=None	0.8030

KNN определяет класс по соседним объектам; веса distance усиливают вклад ближайших соседей. SVM строит разделяющую границу, RBF позволяет ей быть нелинейной; C управляет штрафом за ошибки, gamma — влиянием отдельного объекта. Дерево последовательно делит объекты по условиям на признаки; ограничение глубины и минимальный размер листа сдерживают переобучение.

8 Оценка на тестовой выборке

Ниже precision, recall и F1 относятся к положительному классу — дефолту. ROC-AUC вычислен по вероятности класса 1 для KNN/дерева и decision_function для SVM. Полные отчёты обоих классов находятся в results.json.

В таблице 2 приведены результаты сравнения моделей.

Таблица 2 - Результаты сравнения моделей

Модель	Precision	Recall	F1	ROC-AUC
KNN	0.8815	0.5889	0.7061	0.8771
SVM	0.9120	0.6616	0.7669	0.9010
Дерево решений	0.9759	0.6858	0.8055	0.9092

На рисунке 6 представлен результат этапа: истинные и предсказанные классы.

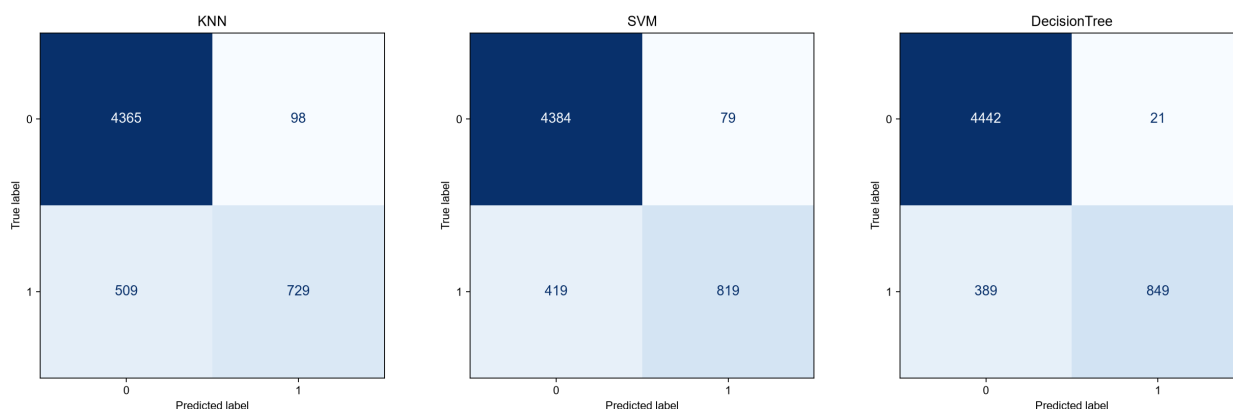


Рисунок 6 - Истинные и предсказанные классы

Строки матрицы соответствуют истинным классам, столбцы — предсказанным. Диагональ — верные ответы. Для дерева: 4 442 верных ответа класса 0, 849 верных дефолтов, 21 ложное предсказание дефолта и 389 пропущенных дефолтов.

На рисунке 7 представлен результат этапа: roc-кривые.

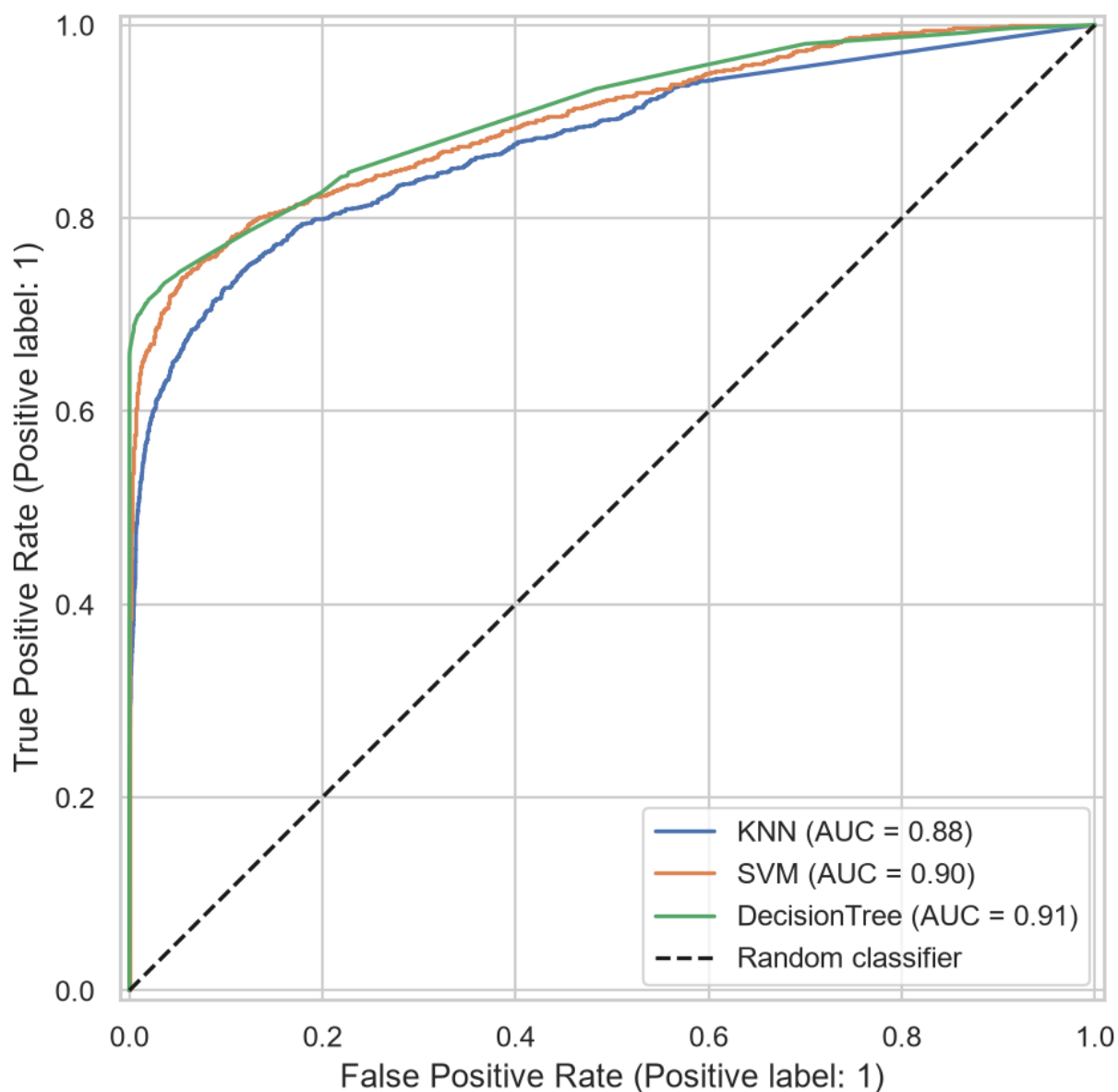


Рисунок 7 - ROC-кривые

ROC-кривая показывает долю найденных дефолтов относительно доли ложных срабатываний при изменении порога. Диагональ соответствует случайному ранжированию. ROC-AUC не заменяет оценку числа пропущенных дефолтов.

9 Визуализация дерева

На рисунке 8 представлен результат этапа: первые четыре уровня дерева.

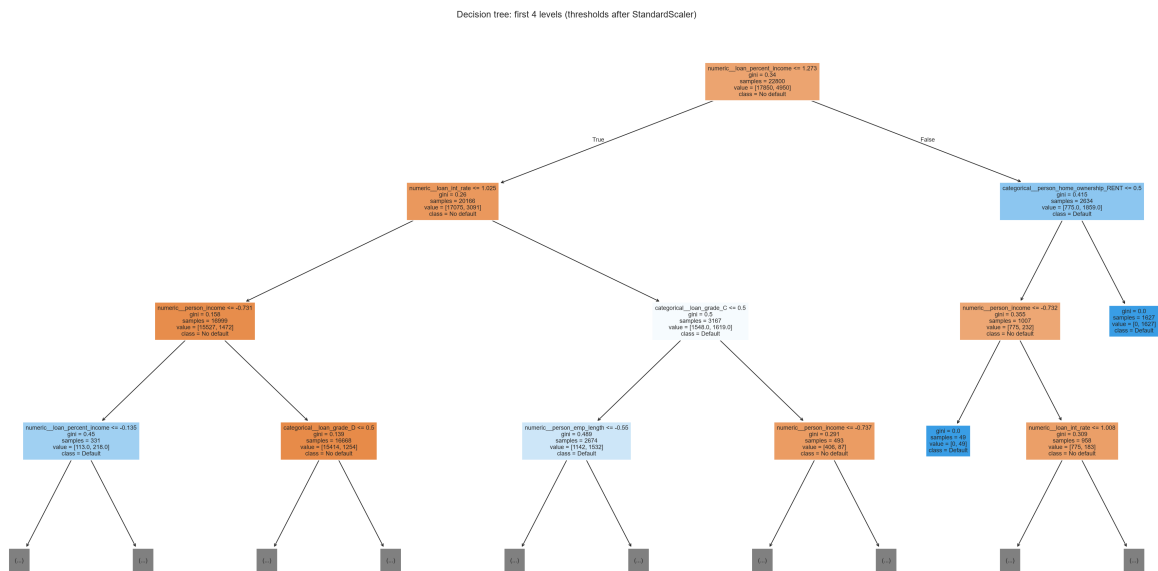


Рисунок 8 - Первые четыре уровня дерева

Полная глубина модели — 8. На рисунке показаны первые четыре уровня; многоточия означают продолжение ветвей. Корневое условие использует отношение суммы кредита к доходу. Затем используются ставка, тип жилья, доход и категория кредита. Числовые пороги заданы после стандартизации. samples — число тренировочных объектов в узле, value — числа объектов классов 0 и 1, gini — мера неоднородности классов (0 означает один класс).

ЗАКЛЮЧЕНИЕ

Цель достигнута: освоен полный процесс классификации от очистки до оценки. Все задачи выполнены: построены графики признаков, удалены пропуски и дубликаты, обучены три метода с подбором параметров, рассчитаны четыре метрики и визуализировано дерево. По CV F1 выбрано дерево; на тесте $F1=0.8055$, $ROC-AUC=0.9092$. Высокий precision 0.9759 сочетается с recall 0.6858: найдено 849 из 1238 дефолтов, 389 пропущены. Вывод о качестве ограничен одной выборкой и сохранёнными аномалиями.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2001. Отчёт о научно-исследовательской работе. Структура и правила оформления. URL: <https://meganorm.ru/Data2/1/4294847/4294847312.htm> (дата обращения: 03.10.2026).
2. Lao Tse. Credit Risk Dataset. Kaggle. URL: <https://www.kaggle.com/datasets/laotse/credit-risk-dataset> (дата обращения: 03.10.2026).
3. Scikit-learn. Руководство пользователя. URL: <https://scikit-learn.org/stable/modules/neighbors.html> (дата обращения: 03.10.2026).
4. Исходный код и сохранённые результаты проекта ОМО. URL: <https://github.com/Gemoroyy1/ОМО> (дата обращения: 03.10.2026).

ПРИЛОЖЕНИЕ А

(обязательное)

Программный код preprocessing.py

Приведён полный исходный текст файла Лаба1/preprocessing.py. Длинные строки перенесены только для печати; исходный файл в репозитории сохраняет исходные переносы и отступы. Код выполнялся в среде Python проекта.

```
"""Учебная предварительная обработка Credit Risk Dataset.
```

```
Запуск: python preprocessing.py
```

```
"""
```

```
import json
```

```
import os
```

```
from pathlib import Path
```

```
ROOT = Path(__file__).resolve().parent
```

```
DATA = ROOT.parent / "data"
```

```
os.environ.setdefault("MPLCONFIGDIR", str(ROOT /  
    ".cache/matplotlib"))
```

```
import matplotlib
```

```
matplotlib.use("Agg") # Сохраняем графики в файлы без открытия  
    окон.
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
import seaborn as sns
```

```
FEATURES = [  
    "person_age", "person_income", "person_emp_length",  
    "loan_amnt",  
    "loan_int_rate", "loan_percent_income",  
    "cb_person_cred_hist_length",  
]
```

```
TARGET = "loan_status"
```

```
def save_figure(path: Path) -> None:
```

```
    plt.tight_layout()
```

```
    plt.savefig(path, dpi=160, bbox_inches="tight")
```

```
    plt.close()
```

```

def visualize(data: pd.DataFrame, directory: Path) -> None:
    """Распределения и связь признаков с дефолтом до/после
    очистки."""
    directory.mkdir(parents=True, exist_ok=True)
    fig, axes = plt.subplots(3, 3, figsize=(15, 12))
    for ax, feature in zip(axes.flat, FEATURES):
        sns.histplot(data=data, x=feature, bins=40, ax=ax)
    for ax in list(axes.flat)[len(FEATURES):]:
        ax.set_visible(False)
    save_figure(directory / "histograms.png")

    fig, axes = plt.subplots(3, 3, figsize=(15, 12))
    for ax, feature in zip(axes.flat, FEATURES):
        sns.boxplot(data=data, x=TARGET, y=feature, ax=ax)
    for ax in list(axes.flat)[len(FEATURES):]:
        ax.set_visible(False)
    save_figure(directory / "boxplots.png")

    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    pairs = [("person_income", "loan_amnt"),
              ("person_age", "person_emp_length"),
              ("loan_int_rate", "loan_percent_income")]
    for ax, (x, y) in zip(axes, pairs):
        sns.scatterplot(data=data, x=x, y=y, hue=TARGET,
                        palette={0: "steelblue", 1:
                                "darkorange"},
                        alpha=0.25, s=12, ax=ax)
    save_figure(directory / "scatterplots.png")

def main() -> None:
    sns.set_theme(style="whitegrid")
    raw = pd.read_csv(DATA / "raw/credit_risk_dataset.csv")
    output = DATA / "processed"
    figures = ROOT / "reports/figures"
    output.mkdir(parents=True, exist_ok=True)
    visualize(raw, figures / "raw")

    complete = raw.dropna().copy()
    clean = complete.drop_duplicates().copy()
    clean.to_csv(output / "credit_risk_clean.csv", index=False)
    visualize(clean, figures / "clean")

    minimum = clean[FEATURES].min()
    maximum = clean[FEATURES].max()
    span = (maximum - minimum).replace(0, 1)

```

```

normalized = clean.copy()
normalized[FEATURES] = (clean[FEATURES] - minimum) / span
normalized.to_csv(output / "credit_risk_normalized.csv",
index=False)

correlation = clean[FEATURES +
[TARGET]].corr(method="pearson")
correlation.to_csv(output / "correlation.csv")
plt.figure(figsize=(11, 9))
sns.heatmap(correlation, annot=True, fmt=".2f",
cmap="coolwarm",
              vmin=-1, vmax=1, center=0, square=True)
save_figure(figures / "correlation_matrix.png")

report = {
    "original_rows": len(raw),
    "missing_by_column": raw.isna().sum().to_dict(),
    "removed_missing_rows": len(raw) - len(complete),
    "removed_duplicate_rows": len(complete) - len(clean),
    "clean_rows": len(clean),
    "normalization_min": minimum.to_dict(),
    "normalization_max": maximum.to_dict(),
    "employment_exceeds_age_rows": int(
        (clean.person_emp_length > clean.person_age).sum()
    ),
}
(output / "preprocessing_report.json").write_text(
    json.dumps(report, ensure_ascii=False, indent=2),
    encoding="utf-8"
)
print(json.dumps(report, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```

ПРИЛОЖЕНИЕ Б

(обязательное)

Программный код classification.py

Приведён полный исходный текст файла Лабa1/classification.py. Длинные строки перенесены только для печати; исходный файл в репозитории сохраняет исходные переносы и отступы. Код выполнялся в среде Python проекта.

```
"""Лабораторная №1: python classification.py."""

import json
import platform

import pandas as pd
import sklearn
from sklearn.compose import ColumnTransformer
from sklearn.metrics import (
    ConfusionMatrixDisplay, RocCurveDisplay,
    classification_report,
    f1_score, precision_score, recall_score, roc_auc_score,
)
from sklearn.model_selection import GridSearchCV,
    StratifiedKFold, train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier, plot_tree

from preprocessing import ROOT, DATA, FEATURES, TARGET, plt,
    save_figure, sns

def make_pipeline(model, categories):
    transform = ColumnTransformer([
        ("numeric", StandardScaler(), FEATURES),
        ("categorical", OneHotEncoder(handle_unknown="ignore",
            sparse_output=False), categories),
    ])
    return Pipeline([("preprocess", transform), ("model",
        model)])
```

```

def main():
    sns.set_theme(style="whitegrid")
    output = ROOT / "reports/lab1"
    output.mkdir(parents=True, exist_ok=True)
    data = pd.read_csv(DATA /
        "raw/credit_risk_dataset.csv").dropna().drop_duplicates()
    x, y = data.drop(columns=TARGET), data[TARGET]
    categories = [column for column in x.columns if column not
        in FEATURES]
    x_train, x_test, y_train, y_test = train_test_split(
        x, y, test_size=0.2, stratify=y, random_state=42
    )
    cv = StratifiedKFold(n_splits=3, shuffle=True,
        random_state=42)
    experiments = {
        "KNN": (KNeighborsClassifier(), {
            "model__n_neighbors": [5, 15, 31], "model__weights":
            ["uniform", "distance"]}),
        "SVM": (SVC(), [
            {"model__kernel": ["linear"], "model__C": [0.1, 1]},
            {"model__kernel": ["rbf"], "model__C": [1, 10],
            "model__gamma": ["scale", 0.01]},
        ]),
        "DecisionTree":
        (DecisionTreeClassifier(random_state=42), {
            "model__max_depth": [4, 8, 12, None],
            "model__min_samples_leaf": [5, 20],
            "model__class_weight": [None, "balanced"]}),
    }
    metrics, details = [], {}
    predictions = pd.DataFrame({"source_row": x_test.index,
        "actual": y_test.to_numpy()})
    fig_cm, axes = plt.subplots(1, 3, figsize=(16, 5))
    fig_roc, ax_roc = plt.subplots(figsize=(8, 6))
    tree_pipeline = None
    for ax, (name, (model, grid)) in zip(axes,
        experiments.items()):
        print(f"Training {name}...", flush=True)
        search = GridSearchCV(make_pipeline(model, categories),
            grid,
            scoring="f1", cv=cv, n_jobs=2,
            error_score="raise")
        search.fit(x_train, y_train)
        best = search.best_estimator_
        predicted = best.predict(x_test)
        score = (best.decision_function(x_test) if name == "SVM"
            else best.predict_proba(x_test)[:, 1])
        metrics.append({

```

```

        "model": name, "cv_f1": search.best_score_,
        "precision": precision_score(y_test, predicted,
zero_division=0),
        "recall": recall_score(y_test, predicted,
zero_division=0),
        "f1": f1_score(y_test, predicted, zero_division=0),
        "roc_auc": roc_auc_score(y_test, score),
    })
    details[name] = {"best_parameters": search.best_params_,
                    "classification_report":
classification_report(
                        y_test, predicted,
output_dict=True, zero_division=0)}
    pd.DataFrame(search.cv_results_).to_csv(output /
f"{name}_cv.csv", index=False)
    predictions[f"{name}_predicted"] = predicted
    predictions[f"{name}_score"] = score
    ConfusionMatrixDisplay.from_predictions(y_test,
predicted, ax=ax,
                                            colorbar=False,
cmap="Blues")
    ax.set_title(name)
    ax.grid(False)
    RocCurveDisplay.from_predictions(y_test, score,
name=name, ax=ax_roc)
    if name == "DecisionTree":
        tree_pipeline = best
        print(metrics[-1], flush=True)
plt.figure(fig_cm.number)
save_figure(output / "confusion_matrices.png")
ax_roc.plot([0, 1], [0, 1], "k--", label="Random
classifier")
ax_roc.legend()
plt.figure(fig_roc.number)
save_figure(output / "roc_curves.png")

plt.figure(figsize=(24, 12))
plot_tree(tree_pipeline.named_steps["model"],
          feature_names=tree_pipeline.named_steps["preproces
s"].get_feature_names_out(),
          class_names=["No default", "Default"],
filled=True,
          max_depth=3, fontsize=8)
plt.title("Decision tree: first 4 levels (thresholds after
StandardScaler)")
save_figure(output / "decision_tree.png")

result = pd.DataFrame(metrics)

```

```

result.to_csv(output / "metrics.csv", index=False)
predictions.to_csv(output / "predictions.csv", index=False)
winner = result.loc[result.cv_f1.idxmax(), "model"]
details["experiment"] = {
    "python": platform.python_version(), "sklearn":
sklearn.__version__,
    "train_rows": len(x_train), "test_rows": len(x_test),
    "train_class_counts": y_train.value_counts().to_dict(),
    "test_class_counts": y_test.value_counts().to_dict(),
    "selected_by_cv_f1": winner, "random_state": 42,
    "tree_depth":
tree_pipeline.named_steps["model"].get_depth(),
}
(output / "results.json").write_text(
    json.dumps(details, ensure_ascii=False, indent=2),
encoding="utf-8")
print(result.to_string(index=False))
print(f"Selected by CV F1: {winner}")

if __name__ == "__main__":
    main()

```